Fanny Saez — Carte de visite web

Dossier de présentation · Entretien de sélection CDA · Simplon Clermont-Ferrand · 24 juin 2026

`HTML5 · CSS3 · ✓ Conforme`

Le projet

J'ai réalisé une **carte de visite au format site web** pour me présenter lors de l'entretien technique. Le site contient deux faces : un **recto** avec mon identité, ma photo et mes centres d'intérêt, et un **verso** avec mes compétences, les raisons pour lesquelles je candidature à la CDA et mes profils en ligne. Il est accessible à l'adresse : **carte-de-visite-fannysaez.vercel.app**

En un coup d'œil

Fichiers HTML	Fichiers CSS	Score Lighthouse	Accessibilité	Déployé sur	Breakpoints
1 fichier `index.html`	3 fichiers `globals · components · utilities`	**93–97 / 100**	**100 / 100**	Vercel en ligne 24h/24	3 Mobile · Tablette · Desktop

Conformité au brief — livrables

	Ce que demande le brief	Comment c'est fait	Dans le code
✓	Un fichier HTML pour la structure	Le fichier *index.html* contient toute la structure de la page	`index.html`
✓	Balise titre — nom et prénom	Balise `<h1>` affiche « Fanny Saez » en grand sur le recto	`<h1>Fanny Saez</h1>`
✓	Balise image — avatar ou photo	Balise `<img>` avec ma photo en format WebP, avec texte alternatif	`<img src="photo-fanny.webp">`
✓	Liste des raisons de rejoindre Simplon	Section « Pourquoi la CDA ? » avec une liste `<ul>` de 4 items : La conception · L'évolution · L'écosystème · Et après ?	`<ul id="simplon-list">`
✓	Liste et liens — profils en ligne	Section « Profils en ligne » : liste `<ul>` avec 6 liens cliquables (GitHub, LinkedIn, Instagram, Portfolio, CV, BambuCalc)	`<ul class="profile-list">`
✓	Un fichier CSS pour la mise en	3 fichiers CSS séparés par rôle : variables/polices, composants, responsive — plus lisible, plus	`styles/*.css`

	Ce que demande le brief	Comment c'est fait	Dans le code
	forme	maintenable	
✓	Au moins 1 règle CSS par balise HTML	Toutes les balises utilisées ont au moins une règle CSS : html, body, h1, h2, p, ul, li, a, img, figure, figcaption, section, strong, button, span, dialog…	`globals.css + components.css`

Fanny Saez — Explication technique

Ce que signifient les termes · Ce que j'ai réalisé · Comment je me suis organisée · 24 juin 2026

Ce que signifient HTML5 et CSS3

H — HTML5 · la structure HTML est le **squelette** du site. Chaque balise est une instruction qui dit au navigateur quoi afficher. Par exemple, `<h1>` signifie « titre principal », `<ul><li>` signifie « liste à puces », `<img>` signifie « affiche une image », `<a>` signifie « lien cliquable ». HTML définit *ce qu'il y a* sur la page, pas comment ça ressemble.

C — CSS3 · la mise en forme CSS est l'**habillage** du site. Il dit comment afficher ce que HTML décrit : couleur, taille, police, espacement, animation, disposition. Une règle CSS ressemble à `h1 { color: teal; font-size: 2rem; }` : on cible une balise et on lui applique des propriétés. CSS3 apporte les *transitions*, les *animations* et la *mise en page en grille*.

≡ — Balise sémantique Une balise est « sémantique » quand son nom dit ce qu'elle contient. `<section>` = une section thématique, `<figure>` = une image avec légende, `<strong>` = texte important. Cela aide les moteurs de recherche et les outils d'accessibilité à comprendre la page.

R — Responsive design Le site s'adapte automatiquement à la taille de l'écran : téléphone, tablette, ordinateur. En CSS, on utilise des `@media` (« règles conditionnelles ») qui activent des styles différents selon la largeur de l'écran. Ma carte a 3 versions : mobile ≤ 600 px, tablette ≤ 768 px, desktop.

Critères d'évaluation — tous satisfaits

- ✓ **Les rôles respectifs d'HTML et de CSS sont explicités** `index.html` = structure uniquement. `styles/*.css` = mise en forme uniquement. Aucun style dans le HTML lui-même.
- ✓ **La structure HTML est fonctionnelle** La page s'affiche correctement dans tous les navigateurs. Déployée en ligne sur Vercel, accessible 24h/24.
- ✓ **La structure HTML est cohérente** Balises sémantiques utilisées partout : `<section>`, `<h1>`/`<h2>`, `<ul>`/`<li>`, `<figure>`/`<figcaption>`, attributs `aria-label` pour l'accessibilité.

- ✓ **Les règles CSS sont fonctionnelles et liées à la page HTML** Chaque classe CSS utilisée dans `index.html` a sa définition dans les fichiers CSS. Toutes les balises ont au moins une règle.
- ✓ **Suit les prescriptions et consignes reçues** Tous les livrables du brief sont présents. J'ai suivi les ressources proposées (OpenClassrooms, MDN) et cherché par moi-même pour aller plus loin.
- ✓ **Organisation des tâches en fonction des contraintes** Maquette Figma réalisée avant le développement. Travail en étapes : recto → verso → responsive → performance. Déploiement continu sur Vercel pour tester à chaque étape.

Comment je me suis organisée — 2 jours

> Lève-tôt de nature, je commence mes journées de travail très tôt le matin. J'ai suivi une progression logique : **concevoir avant de coder**, puis **structurer avant de mettre en forme**.

Jour 1 · Brief · Maquette · HTML

Moment	Étape	Détail
Matin	**Lecture du brief + auto-formation**	Analyse des livrables, exemples fournis, ressources OpenClassrooms / MDN
Matinée	**Maquette Figma**	Recto + Verso sur 3 breakpoints (Desktop / Tablette / Mobile), prototype interactif avec réactions
Après-midi	**Structure HTML — index.html**	Toutes les balises du brief : `<h1>`, `<img>`, `<ul><li>`, `<a>` — sections sémantiques recto + verso

Jour 2 · CSS · JavaScript · Déploiement

Moment	Étape	Détail
Très tôt le matin	**CSS — globals.css**	Variables `:root`, polices auto-hébergées, reset, règles de base sur toutes les balises (body, h1, ul, img, a…)
Matinée	**CSS — components.css**	Flip 3D, colonnes recto/verso, accordéon, animations flottantes, onglets compétences
Après-midi	**CSS — utilities.css (responsive)**	`@media` tablette ≤ 768 px, mobile ≤ 600 px — grille 1 colonne portrait sur mobile
Fin d'après-midi	**JavaScript — script.js**	Flip au clic, accordéon exclusif, onglets avec pagination, modale "Plus d'infos"
Soir	**Tests + déploiement Vercel**	Lighthouse 93–97/100 mobile · 100/100 desktop, optimisations finales, mise en ligne

---

Ce que j'ai fait en plus — par curiosité

Bonus	Description
Flip 3D CSS	La carte se retourne comme une vraie carte de visite grâce aux propriétés CSS *transform* et *backface-visibility*. Aucun JavaScript pour cela.
Accordéon interactif	Les sections du verso s'ouvrent et se ferment au clic. Câblé en JavaScript, accessible au clavier avec attributs ARIA.
QR codes cliquables	Deux QR codes SVG générés localement : l'un pointe vers mon portfolio, l'autre vers mon CV en ligne. Cliquables et scannables.
Polices auto-hébergées	La police Raleway est stockée dans le projet. Pas de requête vers Google Fonts : le site charge plus vite et ne dépend d'aucun service externe.
Score Lighthouse	93–97/100 mobile · 100/100 desktop · 100/100 accessibilité · 100/100 SEO. Le score mobile varie entre 93 et 97 selon la charge du serveur de test — variabilité normale du simulateur Lighthouse, pas du code.
Données structurées	Balise JSON-LD Schema.org intégrée : les moteurs de recherche comprennent que la page décrit une personne (nom, email, téléphone, profils).

Fanny Saez — Extraits de code

HTML · CSS · JavaScript — expliqués en mots simples pour l'entretien · 24 juin 2026

`HTML5 · CSS3 · JS`

Extraits HTML — la structure

HTML · Balise titre — nom et prénom

```
<h1>Fanny <span class="accent">Saez</span></h1>
<p class="tagline">Développeuse Web & Mobile</p>
```

`h1` = balise titre de niveau 1 (le plus important de la page) — le jury voit mon nom en grand. `span` = zone de texte sans sémantique, utilisée ici uniquement pour changer la couleur du nom de famille en CSS. `p` = paragraphe, ici pour le métier.

HTML · Balise image — avatar

```
<img
  src="assets/img/photo-fanny.webp"
  alt="Portrait de Fanny Saez, développeuse"
```

```
    class="avatar"
    width="240" height="240"
  />
```

`img` = balise auto-fermante qui affiche une image. `src` = chemin vers le fichier. `alt` = texte alternatif lu par les lecteurs d'écran (accessibilité). `width/height` = dimensions pour éviter le décalage de page au chargement.

HTML · Liste raisons de rejoindre la CDA

```
<ul id="simplon-list" class="reason-list">
  <li>La conception</li>
  <li>L'évolution</li>
  <li>L'écosystème</li>
  <li>Et après ?</li>
</ul>
```

`ul` = liste non ordonnée (unordered list) — les puces. `li` = chaque élément de liste (list item). C'est la section "Pourquoi la CDA ?" du verso : 4 raisons cliquables avec accordéon.

HTML · Liste profils en ligne avec liens

```
<ul class="profile-list">
  <li>
    <a href="https://github.com/fannysaez"
       target="_blank"
       rel="noopener noreferrer">
      GitHub
    </a>
  </li>
  <li><a href="https://linkedin.com/in/fannysaez" ...>LinkedIn</a></li>
  <!-- + Instagram, Portfolio, CV, BambuCalc -->
</ul>
```

`a` = lien hypertexte (anchor). `href` = adresse cible. `target="_blank"` = ouvre dans un nouvel onglet. `rel="noopener noreferrer"` = sécurité (évite que la page cible accède à notre onglet).

Extraits CSS — la mise en forme

CSS · Règle sur h1

```
h1 {
  font-size: clamp(2.3rem, 5vw, 3.2rem);
  font-weight: 800;
  color: var(--white);
```

```
  line-height: 1.1;
}
```

`clamp()` = taille fluide : jamais moins de 2.3rem, jamais plus de 3.2rem. S'adapte à la taille d'écran automatiquement. `var(--white)` = variable CSS définie une seule fois dans `:root`.

CSS · Règles sur ul, li, a, img

```
ul  { list-style: none; }
li  { padding: 0; }
a   { text-decoration: none; }
img {
  display: block;
  max-width: 100%;
}
```

Le **reset CSS** : on enlève les styles par défaut du navigateur pour repartir d'une base propre. `list-style: none` = supprime les puces. `max-width: 100%` = l'image ne déborde jamais de son conteneur.

CSS · Responsive — @media mobile

```
/* S'applique si écran ≤ 600 px */
@media (max-width: 600px) {
  h1 {
    font-size: 1.75rem;
    text-align: center;
  }
  .avatar-wrap {
    width: 120px;
    height: 120px;
  }
}
```

`@media` = règle conditionnelle. Le CSS à l'intérieur ne s'applique que si la condition est vraie (ici : écran ≤ 600 px). C'est comme ça qu'on rend un site responsive sans JavaScript.

CSS · Flip 3D de la carte

```
.card-3d {
  transform-style: preserve-3d;
  transition: transform 0.82s ease;
}
.card-3d.is-flipped {
  transform: rotateY(180deg);
```

```
}
.face--back {
  transform: rotateY(180deg);
  backface-visibility: hidden;
}
```

Le retournement est 100 % CSS. `preserve-3d` = les enfants vivent dans l'espace 3D. `backface-visibility: hidden` = on cache la face quand elle est dos à l'écran. La classe `.is-flipped` est ajoutée par JavaScript au clic.

CSS · Variables et couleurs

```
:root {
  --teal:    #1ebfc1;
  --dark:    #08131e;
  --white:   #e6f2fb;
  --muted:   #567585;
}
strong { color: var(--teal); }
body   { background: var(--dark); }
```

`:root` = la racine du document. On y déclare les **variables CSS** avec `--nom`. On les réutilise partout avec `var(--nom)`. Changer une couleur ici la change sur tout le site.

CSS · Animation flottante (badges passions)

```
@keyframes pb-float {
  0%, 100% { transform: translateY(0); }
  50%      { transform: translateY(-10px); }
}
.passion-badge {
  animation: pb-float 4.2s ease-in-out infinite;
}
```

`@keyframes` = on définit une animation image par image. Ici, les icônes de passions montent et descendent en boucle. `ease-in-out` = démarrage et arrêt progressifs (naturel). `infinite` = tourne en boucle.

Extraits JavaScript — les comportements (bonus)

JS · Retourner la carte au clic

```
// On sélectionne la carte dans le HTML
const card = document.querySelector('#card');
```

```
// On écoute le clic (et la touche Entrée)
card.addEventListener('click', () => {
  card.classList.toggle('is-flipped');
});

card.addEventListener('keydown', (e) => {
  if (e.key === 'Enter' || e.key === ' ')
    card.classList.toggle('is-flipped');
});
```

`querySelector` = sélectionner un élément HTML depuis JS. `addEventListener` = écouter un événement (clic, touche...). `classList.toggle` = ajouter la classe CSS si absente, la retirer si présente. C'est ce toggle qui déclenche le flip 3D CSS.

JS · Ouvrir / fermer un item accordéon

```
// Pour chaque bouton "+" dans les listes
const toggles = document.querySelectorAll('.accordion-toggle');

toggles.forEach(btn => {
  btn.addEventListener('click', (e) => {
    e.stopPropagation(); // ne pas retourner la carte
    const body = btn.closest('.acc-item')
                    .querySelector('.acc-body');
    const isOpen = btn.getAttribute('aria-expanded') === 'true';
    btn.setAttribute('aria-expanded', String(!isOpen));
    body.classList.toggle('hidden', isOpen);
    btn.textContent = isOpen ? '+' : '−';
  });
});
```

`querySelectorAll` = sélectionner plusieurs éléments (renvoie une liste). `forEach` = faire la même chose pour chacun. `stopPropagation` = empêcher le clic de « remonter » jusqu'à la carte (sinon elle se retournerait). `aria-expanded` = attribut d'accessibilité : informe les lecteurs d'écran si ouvert ou fermé.
